

HydIR

From machine bytes to inspectable behavior

Binary lifting workbench / project deep dive

September 2026

| What can an executable tell us?

01 Where can execution go?

02 What values decide each branch?

03 Can a changed binary keep the behavior we expect?

Why lift machine code?

```
movq %rdi, %rax  
cmpq %rsi, %rdi  
jae .Ldone  
movq %rsi, %rax  
.Ldone:  
ret
```

Assembly makes the machine explicit

A compare changes flags. A later branch reads those flags. Register writes and addresses carry the rest of the story.

Lifting translates these effects into a form that analysis and transformation tools can handle.

| Why LLVM IR?

Explicit data flow

- SSA values expose how results depend on inputs and earlier blocks.

Reusable transformations

- LLVM passes simplify instructions and control flow after semantics are encoded.

A checkable artifact

- The verifier catches malformed IR before the next experiment or build.

Striga's teaching example

x86 registers + flags $\rightarrow$ State* + guest memory $\rightarrow$ LLVM IR

Striga models register and flag effects in a State structure. Its article uses LLVM optimization to remove bookkeeping after an ABI wrapper restores function arguments and returns.

When the bookkeeping disappears

Lifted shape (conceptual)

```
state.rdi += state.rsi;  
state.rax = state.rdi;  
return state.rax;
```

After an ABI wrapper and LLVM passes

```
uint64_t add(uint64_t x,  
uint64_t y) {  
    return x + y;  
}
```

The point: machine-state scaffolding can become ordinary data flow once its calling convention is made explicit.

HydIR takes a narrower route

Bounded ELF function



Reachable CFG



SSA values for registers + flags

HydIR's scalar path emits per-block ϕ values and explicit branch conditions. It accepts a selected instruction subset and rejects unknown memory, calls, indirect edges, and partial registers.

One function, several artifacts

ELF bytes → CFG → LLVM IR → C / passes / rebuild

Each step has a different purpose: inspect the binary, recover control flow, encode semantics, then choose an output or experiment.

The scalar lift contract

Required

Little-endian x86-64 ELF. A chosen symbol or explicit entry and size. An analyst assertion of `u64(u64, u64)`.

Accepted semantics

Full-width scalar moves and arithmetic, compare/test, returns, and direct conditional branches. Returning paths define RAX.

Anything outside the contract is rejected instead of guessed.

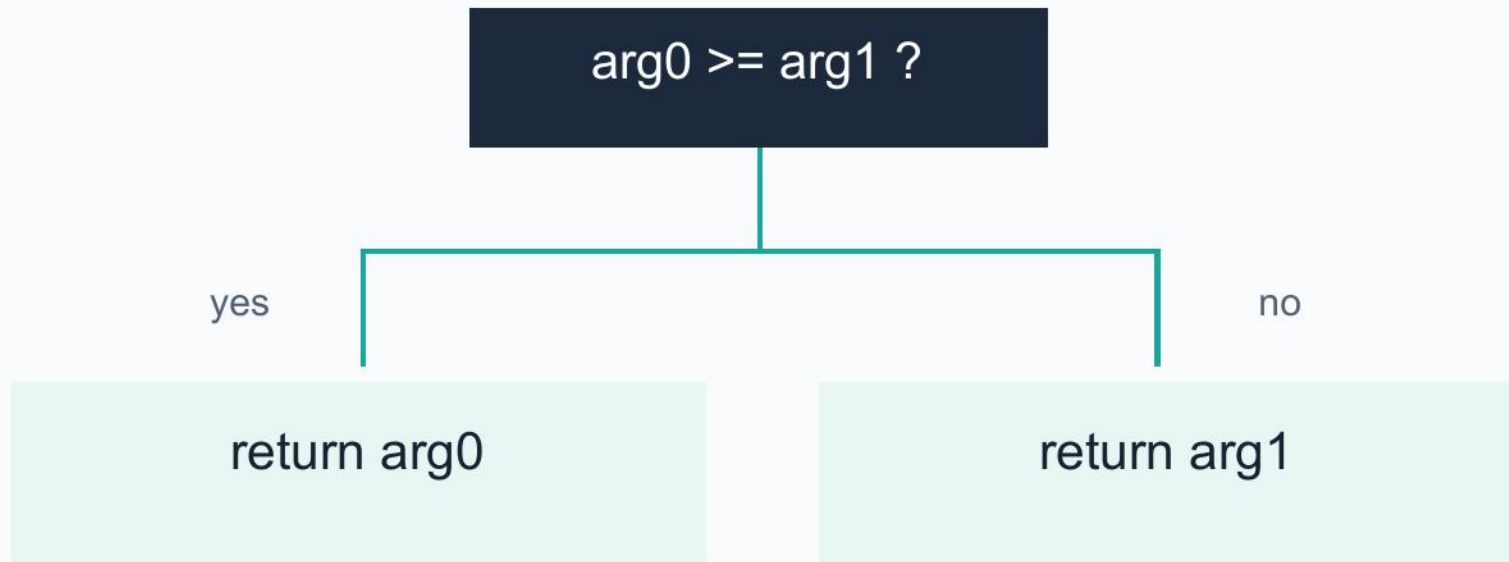
Worked example: hydir_max2

```
hydir_max2:  
movq %rdi, %rax  
cmpq %rsi, %rdi  
jae .Ldone  
movq %rsi, %rax  
.Ldone:  
ret
```

What the branch means

The compare sets flags for unsigned RDI versus RSI. JAE keeps arg0 when it is at least arg1. Otherwise, RSI replaces the return value in RAX.

The branch is the behavior



Flags become explicit data flow

```
%cf = icmp ult i64 %arg0, %arg1
%keep_arg0 = xor i1 %cf, true
br i1 %keep_arg0, label %ret,
label %use_arg1
```

Unsigned comparison

The carry flag becomes a boolean value. The conditional branch names both successors.

Readable excerpt from HydlR's emitted IR. Variable names are shortened here.

Scalar C is another view

```
uint64_t hydir_lifted(  
    uint64_t arg0,  
    uint64_t arg1) {  
    return arg0 >= arg1  
        ? arg0 : arg1;  
}
```

What it communicates

The unsigned max idiom reads clearly here. The CFG and IR still preserve the lower-level branch and flag evidence behind it.

High-level structured C recovery is not implemented yet.

LLVM IR enables controlled experiments

raw.ll  **verified before.ll**  **verified after.ll**

InstCombine

simplify instructions

SCCP

propagate
constants

SimplifyCFG

merge and prune
blocks

DCE

remove dead
work

An IR change and a verifier pass do not prove behavior stayed the same.

Verification needs an independent check

Native executable

Lifted LLVM / compiled C

same chosen inputs

Compare observable results

For the scalar demos, HydIR compares results against native execution on Linux x86-64.

20

distinct supported scalar
functions

1,008

inputs per function in the local
demos

The corpus covers a deliberate scalar subset. It does not establish general binary correctness.

A workbench for inspecting the result

Navigator

Pick symbols and functions

Graph

Follow reachable blocks

IR + C

Read lifted artifacts

Console

Review warnings and reports

Local ELF opening stays local. Uploading a binary requires a separate labelled action.

Two project contexts

Local

Private on-device SQLite ledger.
Path-bound binary identity. Analyst
names, comments, and scoped
assumptions.

Remote

Authenticated loopback service.
Explicit upload creates an immutable
binary revision. Lift jobs can be
monitored and cancelled.

The ledgers do not automatically sync.

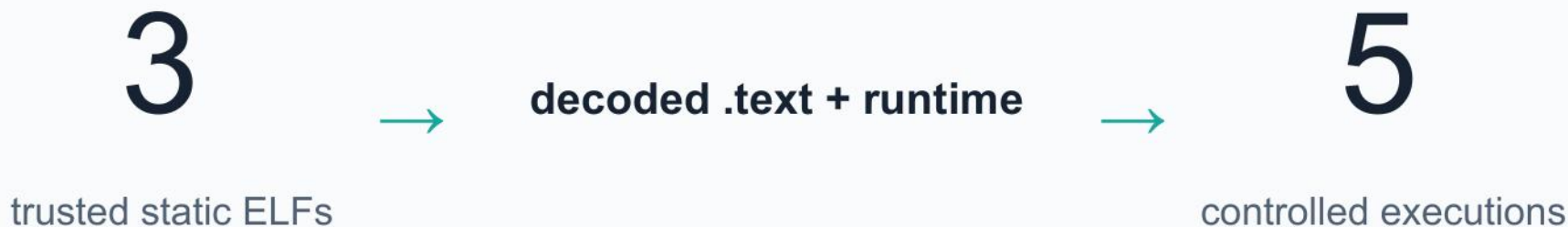
Changes leave a trail

Original ELF → **IR snapshots / patch** → **New revision or file**

Named pass experiments retain raw, before, and after IR. Remote edits create new project revisions. Local patches and rebuilds write new outputs.

Analyst comments and assumptions are labelled unverified.

A bounded whole-executable path



The demo compares stdout, stderr, and exit status. Unsupported instructions and OS behavior are rejected. This is not general ELF recompilation.

A concrete showcase

HYDR

original

ACCESS GRANTED

same input

after scalar patch

ACCESS DENIED

rebuilt

HYDR + NOPE

matches original

The showcase script checks these cases on a trusted fixture; the access phrase is a demo, not an authentication system.

Where HydlR stops today

No automatic stripped-code discovery

No high-level structured C recovery

No general hostile-binary sandbox or equivalence proof

The current value is a tested, inspectable vertical slice with explicit rejection.

Sources and discussion

Project evidence

README.md
tests/fixtures/max2.S
target/demo-showcase/max2.ll
scripts/demo-passes.sh
scripts/demo-showcase.sh

Reading

mrexodia, Striga: Lifting x86 to LLVM IR
with Python
secret.club/2026/05/21/striga.html

LLVM Language Reference and Passes
llvm.org/docs/

HydIR is an early project. These slides describe the checked subset in this repository.